答案與預期輸出

> 每個單元都附「跑出來應該長什麼樣」，以及解題的關鍵判斷。 完整程式碼在 `answer/` 資料夾。**先自己寫完再看。**

怎麼對答案

```
cd practice
pytest -v test_ul_parse.py      # 跑你寫的
```

```
cd answer
pytest -v test_ul_parse.py      # 跑參考解答
```

比對重點不是程式碼一不一樣，而是： 1. 最後一行的數字對不對得上 2. 該失敗的有沒有失敗、該通過的有沒有通過 3. 你能不能說出每個測試在驗證什麼

全部答案一次跑

```
1 failed, 52 passed, 1 skipped, 2 xfailed
```

結果	來自	正常嗎
1 failed	Unit 2 的產品 bug	✅ 正常，那是故意留的 bug
1 skipped	Unit 6 的 Linux 專用測試	✅ 在 Windows 上會跳過
2 xfailed	Unit 6 的已知誤報與漏報	✅ 預期中的失敗

Unit 1｜新日誌來源上線前的驗證

預期輸出

```
test_ul_parse.py::test_basic_fields_parsed PASSED
test_ul_parse.py::test_ts_converted_to_float PASSED
test_ul_parse.py::test_missing_user_is_none PASSED
test_ul_parse.py::test_int_port_stays_int PASSED
test_ul_parse.py::test_ipv6_source_supported PASSED

============================== 5 passed ==============================
```

解題重點

```
def test_ts_converted_to_float():
    e = parse_event(SAMPLE)
    assert isinstance(e["ts"], float)
    assert e["ts"] == 1700000000.0
```

型別也是規格的一部分。 只寫 `assert e["ts"] == 1700000000` 會通過，因為在 Python 裡 `1700000000 == 1700000000.0` 是 True —— 但這樣就沒驗證到「轉成 float」這件事。

```
def test_missing_user_is_none():
    e = parse_event('{"ts": 1, "src_ip": "45.67.89.10", "action": "conn_blocked"}')
    assert e["user"] is None
```

用 `is None` 而不是 `== None`。這是 Python 的慣例，也比較嚴謹。

Unit 2 | CI 亮紅燈，你負責分診

修改前（原本的四個測試）

```
========================= 2 failed, 2 passed =========================
```

修改後的預期輸出

```
========================= 1 failed, 3 passed =========================
```

還剩 1 個失敗是對的。 那是產品 bug，不能為了讓 CI 變綠去改它。

分診結果

測試	失敗訊息	判斷	依據
`test_action_converted_to_uppercase`	`assert 'login_failed' == 'LOGIN_FAILED'`	**測試寫錯**	規格寫「action 原樣保留，不轉大小寫」
`test_string_port_converted_to_int`	`assert '443' == 443`	**產品 bug**	規格寫字串 "443" 必須轉成整數 443

那個 bug 的失敗訊息

```
    def test_string_port_converted_to_int():
        e = parse_event('{"ts": 1, "src_ip": "45.67.89.10", "action": "conn_blocked", "dst_port": "443"}')
>       assert e["dst_port"] == 443
E       AssertionError: assert '443' == 443
```

注意 `'443'` 有引號、`443` 沒有。**一個是字串，一個是整數。** 這種錯誤肉眼很難看出來，pytest 的訊息直接點出來了。

Bug 回報範例

```
標題：parse_event 收到字串型 dst_port 時未轉為整數
severity：中
重現步驟：
  parse_event('{"ts":1,"src_ip":"45.67.89.10","action":"conn_blocked","dst_port":"443"}')
預期結果：dst_port == 443（int），依 parse_event 規格
實際結果：dst_port == "443"（str）
影響範圍：
  下游若用 dst_port 比對（例如「連到 22 或 3389 的連線」），
  "443" == 443 為 False，規則會靜默失效 → 可能造成漏報。
  只影響會送字串 port 的設備。
重現測試：test_u2_triage.py::test_string_port_converted_to_int
```

好的 bug 回報要讓開發者不用問你任何問題就能重現。「影響範圍」那一段最有價值 —— 它說明了這個 bug 為什麼值得修。

> 💡 **延伸**：學到 Unit 6 之後，這個測試可以標上 `@pytest.mark.xfail(reason="BUG-xxx")`，讓它留在套件裡追蹤，又不會讓 CI 一直紅。

Unit 3 | 客戶環境的髒資料

預期輸出

```
test_u3_malformed.py::test_non_json_raises PASSED
test_u3_malformed.py::test_missing_src_ip_raises PASSED
test_u3_malformed.py::test_invalid_ip_raises PASSED
test_u3_malformed.py::test_port_out_of_range_raises PASSED
test_u3_malformed.py::test_unknown_action_raises PASSED
test_u3_malformed.py::test_batch_skips_bad_lines PASSED
test_u3_malformed.py::test_all_bad_lines_no_exception PASSED

============================== 7 passed ==============================
```

解題重點

```
def test_missing_src_ip_raises():
    with pytest.raises(MalformedEvent, match="src_ip"):
        parse_event('{"ts": 1, "action": "login_failed"}')
```

`match="src_ip"` 確保錯誤訊息**指出了是哪個欄位**。 如果只寫 `pytest.raises(MalformedEvent)`，就算程式回報「缺少欄位: ts」也會通過。

```
def test_batch_skips_bad_lines():
    lines = [
        GOOD,                                  # 第 1 行
        "亂碼@@@",                             # 第 2 行 ✗
        GOOD,                                  # 第 3 行
        "",                                    # 第 4 行 空白，不算
        json.dumps({"ts": 1, "action": "x"}),  # 第 5 行 ✗ 缺 src_ip
        GOOD,                                  # 第 6 行
    ]
    result = ingest_batch(lines)

    assert len(result.events) == 3
    assert result.rejected == 2
    assert "第 2 行" in result.reasons[0]
    assert "第 5 行" in result.reasons[1]
```

在每一行旁邊註明行號和預期，未來讀測試的人一看就懂。

```
def test_all_bad_lines_no_exception():
    result = ingest_batch(["???", "{", "not json at all"])
    assert result.events == []
    assert result.rejected == 3
```

這題**沒有**用 `pytest.raises`。如果 `ingest_batch` 丟了例外，測試會直接 ERROR —— 這本身就是檢查。

為什麼未知的 action 不能默默收下

偵測規則只認得清單內的 action。收下一個沒人認得的事件，等於讓它**從所有規則旁邊溜過去**。 與其悄悄放行，不如明確拒絕並記錄，讓人知道有新格式需要處理。

Unit 4 | 測試一條有「記憶」的偵測規則

預期輸出

```
test_u4_fixture.py::test_four_failures_no_alert PASSED
test_u4_fixture.py::test_five_failures_one_alert PASSED
test_u4_fixture.py::test_engine_is_fresh_each_test PASSED
test_u4_fixture.py::test_different_ips_counted_separately PASSED
test_u4_fixture.py::test_success_clears_failures PASSED
test_u4_fixture.py::test_no_duplicate_alert_in_cooldown PASSED

============================== 6 passed ==============================
```

conftest.py 解答

```
@pytest.fixture
def engine():
    eng = DetectionEngine([BruteForceRule(threshold=5, window=300)])
    yield eng
    eng.reset()


@pytest.fixture
def make_event():
    def _make(ts, ip="45.67.89.10", action="login_failed", user="alice"):
        return {"ts": float(ts), "src_ip": ip, "action": action,
                "user": user, "dst_port": 22}
    return _make
```

`make_event` **是工廠型 fixture**。它不回傳事件，而是回傳「做事件的函式」。這樣每個測試都能用一行做出自己需要的事件，只寫跟預設值不同的參數：

```
make_event(ts=0)                                    # 全用預設
make_event(ts=10, user="admin")                     # 只改帳號
make_event(ts=20, ip="91.200.12.34", action="login_success")
```

解題重點

```
def test_engine_is_fresh_each_test(engine):
    assert engine.alerts == []
    assert engine.events_seen == 0
```

這題刻意排在「產生告警」那題之後。fixture 寫對，它就通過；如果你把 scope 改成 `module`，它會失敗，因為上一題的告警還留著。

```
def test_success_clears_failures(engine, make_event):
    for t in range(4):
        engine.feed(make_event(ts=t))
    engine.feed(make_event(ts=4, action="login_success"))
    engine.feed(make_event(ts=5))
    assert engine.alerts == []
```

這條規則的漏洞

攻擊者如果手上有一組有效帳密，可以**每試 4 次就登入一次**，計數一直被清零，永遠不會觸發告警。

你寫測試的時候發現了規則的設計缺陷 —— 這正是測試工程師最有價值的時刻。 實務上你會把它寫進回報，交給 Detection Engineering 團隊判斷要不要調整。

Unit 5 | 誤報與漏報的分界線

預期輸出

```
test_u5_boundary.py::test_failure_threshold[3-False] PASSED
test_u5_boundary.py::test_failure_threshold[4-False] PASSED
test_u5_boundary.py::test_failure_threshold[5-True] PASSED
test_u5_boundary.py::test_failure_threshold[6-True] PASSED
test_u5_boundary.py::test_failure_threshold[10-True] PASSED
test_u5_boundary.py::test_time_window_boundary[240-True] PASSED
test_u5_boundary.py::test_time_window_boundary[299-True] PASSED
test_u5_boundary.py::test_time_window_boundary[300-True] PASSED
test_u5_boundary.py::test_time_window_boundary[301-False] PASSED
test_u5_boundary.py::test_time_window_boundary[600-False] PASSED
test_u5_boundary.py::test_severity_by_user[admin] PASSED
test_u5_boundary.py::test_severity_by_user[uppercase_ROOT] PASSED
test_u5_boundary.py::test_severity_by_user[Administrator] PASSED
test_u5_boundary.py::test_severity_by_user[regular_user] PASSED
test_u5_boundary.py::test_severity_by_user[no_user] PASSED
test_u5_boundary.py::test_severity_by_user[trap_admin_bob] PASSED
test_u5_boundary.py::test_non_failure_actions_not_counted[login_success] PASSED
test_u5_boundary.py::test_non_failure_actions_not_counted[conn_blocked] PASSED
test_u5_boundary.py::test_non_failure_actions_not_counted[conn_allowed] PASSED

============================== 19 passed ==============================
```

解題重點

次數門檻：邊界在 4 / 5

```
@pytest.mark.parametrize("count, should_alert", [
    (3, False),
    (4, False),     # 邊界前：最後一個不告警
    (5, True),      # 邊界：第一個告警
    (6, True),      # 邊界後
    (10, True),
])
```

如果有人把程式的 `<` 改成 `<=`，門檻就從 5 變成 6，`(5, True)` 這組會立刻失敗。

時間窗：邊界在 300 / 301

span	5 次失敗的時間點	告警？	為什麼
300	0, 75, 150, 225, **300**	✅	最新減最舊剛好 300，規格說仍在窗內
301	0, 75.25, 150.5, 225.75, **301**	❌	301 − 0 > 300，第一筆被擠出窗外，只剩 4 次

這種「差 1 秒」的邊界，就是誤報和漏報的分界線。

嚴重度的陷阱：`admin_bob`

```
("admin_bob", "medium"),      # 只是「開頭像」admin，不是特權帳號
```

規格寫的是「帳號**是** admin / root / administrator」，不是「帳號**包含** admin」。如果程式寫成 `"admin" in user`，`admin_bob` 就會被誤判成高風險 —— 這題就是在抓這種 bug。

等價類

```
@pytest.mark.parametrize("action", ["login_success", "conn_blocked", "conn_allowed"])
```

這三種 action 對規則來說都是「不計數」，屬於同一個等價類。每種都測一次，確認沒有哪一種被誤算進去。

Unit 6｜告警品質回歸套件

預期輸出

```
test_u6_markers.py::test_50k_events_within_3_seconds PASSED
test_u6_markers.py::test_service_account_no_alert XFAIL (已知誤報：svc...)
test_u6_markers.py::test_distributed_brute_force_detected XFAIL (已知漏報：...)
test_u6_markers.py::test_linux_log_dir_exists SKIPPED (Linux 系統...)
test_u6_markers.py::test_basic_brute_force_detection PASSED
test_u6_markers.py::test_successful_login_no_alert PASSED

================== 3 passed, 1 skipped, 2 xfailed ==================
```

篩掉慢的： `pytest -v -m "not slow" test_u6_markers.py`

```
============ 2 passed, 1 skipped, 1 deselected, 2 xfailed ============
```

只跑偵測相關： `pytest -v -m "detection" test_u6_markers.py`

```
============ 2 passed, 2 deselected, 2 xfailed ============
```

> 為什麼 `-m detection` 有 2 passed 加 2 xfailed？因為四個標了 `detection` 的測試裡， 兩個是正常的、兩個是 xfail。效能測試和 Linux 測試沒標，所以被 deselect。

conftest.py 解答

```
def pytest_configure(config):
    config.addinivalue_line("markers", "detection: 偵測規則相關的測試")
    config.addinivalue_line("markers", "slow: 跑比較久的測試，開發時可跳過")
```

解題重點

```
@pytest.mark.detection
@pytest.mark.xfail(reason="已知誤報：svc_ 服務帳號的重試會觸發暴力破解告警，預計下版加入例外")
def test_service_account_no_alert(engine, make_event):
    ...
```

reason 要寫到三個月後的人看得懂：是什麼問題、為什麼還沒修、什麼時候會修。

```
@pytest.mark.skipif(sys.platform == "win32", reason="Linux 系統日誌路徑，Windows 上無意義")
```

做完想一想的答案

1. 為什麼用 xfail 不用 skip？ xfail 會持續執行。哪天有人修好了，它會變成 XPASS 提醒你。skip 就永遠不會知道。

2. 修好之後會變成什麼？ XPASS。這時候就該把 xfail 標記拿掉，讓它變成普通的回歸測試，防止日後又壞掉。

> 想讓 XPASS 直接算失敗、強迫大家拿掉標記，可以寫 `xfail(strict=True)`。

3. 誤報和漏報哪個傷害大？ 沒有標準答案，但可以這樣回答： - **漏報**的傷害是直接的 —— 被入侵了卻不知道 - **誤報**的傷害是間接的 —— 告警太多，分析師疲乏（alert fatigue），真的攻擊反而被淹沒

所以兩個都要管，而且調整一個往往會影響另一個。這就是為什麼要有測試把邊界釘住。

Unit 7 | 自動封鎖，但絕對不能封錯

預期輸出

```
test_u7_responder.py::test_high_severity_blocks PASSED
test_u7_responder.py::test_medium_severity_only_notifies PASSED
test_u7_responder.py::test_allowlisted_ip_never_blocked PASSED
test_u7_responder.py::test_private_ip_never_blocked[10.0.0.5] PASSED
test_u7_responder.py::test_private_ip_never_blocked[172.16.0.9] PASSED
test_u7_responder.py::test_private_ip_never_blocked[192.168.1.20] PASSED
test_u7_responder.py::test_firewall_error_does_not_crash PASSED
test_u7_responder.py::test_firewall_rejects PASSED
test_u7_responder.py::test_end_to_end_pipeline PASSED

============================== 9 passed ==============================
```

假防火牆解答

```
class FakeFirewall:
    """假防火牆：記錄每次呼叫，並可模擬三種結果。"""

    def __init__(self, mode="ok"):
        self.mode = mode          # "ok" / "reject" / "error"
        self.calls = []

    def block(self, ip, reason):
        self.calls.append((ip, reason))
        if self.mode == "error":
            raise FirewallError("連線逾時")
        if self.mode == "reject":
            return {"ok": False}
        return {"ok": True}
```

一個類別，三種模式。 測試只要在建立時指定 `mode`，就能模擬各種狀況。

解題重點

```
def test_medium_severity_only_notifies():
    fw = FakeFirewall()
    result = Responder(fw).handle(make_alert(severity="medium"))

    assert result == "notified"
    assert fw.calls == []        # ⭐ 檢查「沒有發生的事」
```

這一行是整個單元最重要的一行。 只檢查回傳值 `"notified"` 是不夠的 —— 程式可能回傳了 "notified"，卻還是偷偷呼叫了防火牆。對自動封鎖功能來說，「不該做的事沒有做」跟「該做的事有做」一樣重要。

```
def test_firewall_error_does_not_crash():
    fw = FakeFirewall(mode="error")
    responder = Responder(fw)
    alert = make_alert()

    result = responder.handle(alert)        # 沒有 pytest.raises

    assert result == "block_failed"
    assert alert in responder.pending_review
```

注意**沒有** `pytest.raises`。規格要求例外不能往外丟，所以如果程式丟了例外，測試會 ERROR，這就是檢查。

為什麼內網 IP 不能自動封鎖

內網 IP 通常是客戶自己的機器 —— 可能是被入侵的員工電腦，也可能是 AD 伺服器。 自動封鎖可能讓整個內網服務中斷，傷害比攻擊本身還大，必須交給人判斷。

端到端測試

```
def test_end_to_end_pipeline():
    lines = [
        json.dumps({"ts": t, "src_ip": PUBLIC_IP, "action": "login_failed", "user": "admin"})
        for t in range(6)
    ]
    lines.insert(3, "%%%亂碼%%%")

    batch = ingest_batch(lines)
    engine = DetectionEngine()
    alerts = engine.feed_many(batch.events)
    fw = FakeFirewall()
    responder = Responder(fw)
    results = [responder.handle(a) for a in alerts]

    assert batch.rejected == 1          # 亂碼被擋下
    assert len(alerts) == 1             # 6 次失敗只告警 1 次（不洗版）
    assert results == ["blocked"]       # admin → high → 封鎖
    assert fw.calls[0][0] == PUBLIC_IP
```

前面每個單元都只測一個元件，這題把它們**串起來**。單元測試全過，不代表接起來就會對 —— 例如 `parse_event` 輸出的欄位名稱如果跟 `BruteForceRule` 期待的不一樣，只有這種測試抓得到。

總檢查表

單元	預期結果	關鍵能力
1	5 passed	用 AAA 寫基本測試，連型別一起驗
2	3 passed, 1 failed	對照規格分診，寫 bug 回報
3	7 passed	`pytest.raises` + `match`，驗證系統韌性
4	6 passed	fixture、yield、工廠型 fixture、測試隔離
5	19 passed	parametrize、等價類、邊界值
6	3 passed, 1 skipped, 2 xfailed	marker、skip vs xfail、回歸追蹤

單元	預期結果	關鍵能力
7	9 passed	假物件、檢查互動、錯誤處理、端到端

全部做完，你可以在簡報第 16 頁「我正在補的能力」的 pytest 卡片上，誠實寫上「已完成實作」。